

MIS 443 – FINAL EXAMINATION

Finance Analysis with PostgreSQL

Course	MIS 443	Business Data Management
Duration	90 minutes	Maximum mark: 100

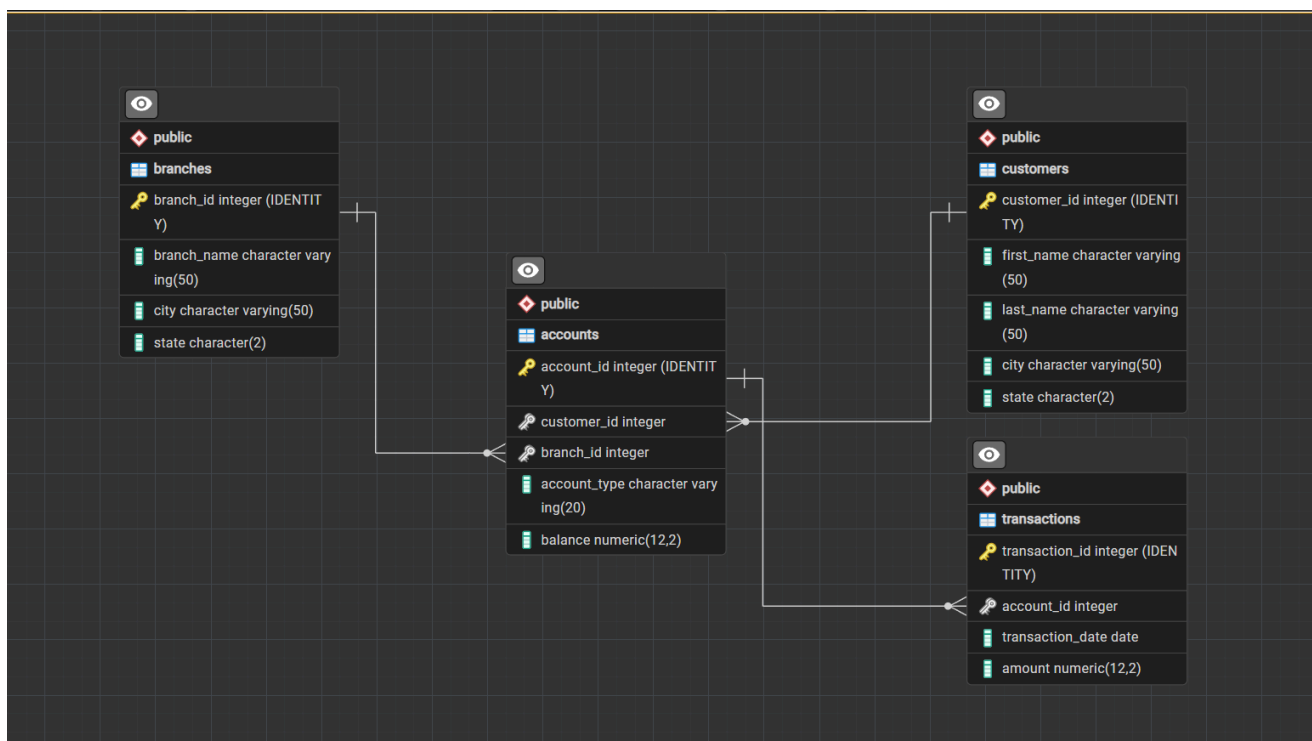
Dataset Introduction

The dataset represents a retail bank that manages customers, branches, bank accounts, and financial transactions. Managers use the data to analyse customer portfolios, account balances, branch performance, and transaction activity.

- Customers: stores customer names and locations.
- Branches: stores branch names and locations.
- Accounts: stores account ownership, branch assignment, account type, and current balance.
- Transactions: stores the date and amount of each account transaction.

Each account belongs to one customer and one branch, and each account may have multiple transactions. Positive account balances represent funds held by customers; negative Credit Card balances represent amounts owed. Positive transaction amounts are credits, while negative amounts are debits.

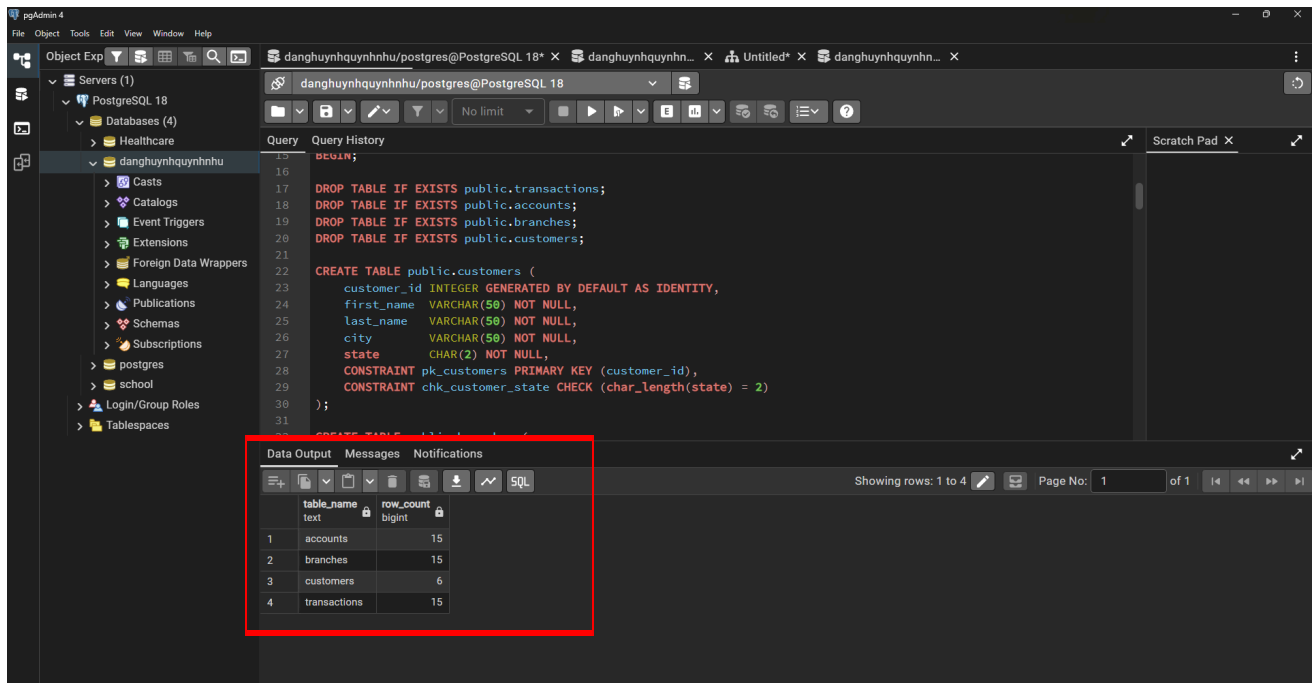
Entity–Relationship Diagram



Examination Questions

Question 1 – Database Setup (10 marks)

Create a PostgreSQL database using your full name in lowercase, without spaces or Vietnamese diacritics. Connect to the database and execute MIS443_Finance_PostgreSQL.sql. Confirm that the customers, branches, accounts, and transactions tables are available in the public schema.



Question 2 – Customer and Account Overview (10 marks)

(a) The Customer Service Manager needs a contact list of customers living in New York. Display customer_id, the customer's full name as full_name, and city. Sort by customer_id. (5 marks)

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including schemas, aggregates, collations, domains, FTS configurations, dictionaries, parsers, templates, foreign tables, functions, materialized views, operators, procedures, sequences, tables, trigger functions, types, views, subscriptions, and roles. The main query editor shows the following SQL code:

```

/*
QUESTION 2 - CUSTOMER AND ACCOUNT OVERVIEW (10 marks)

(a) The Customer Service Manager needs a contact list of customers living in
New York. Display customer_id, full_name, and city, sorted by customer_id.
(5 marks)
*/

-- Your answer here
select customer_id, first_name || ' ' || last_name as full_name, city
from customers
where city = 'New York'
order by customer_id;

```

The query results are displayed in the Data Output tab, showing two rows:

customer_id	full_name	city
1	JohnDoe	New York
2	JaneDoe	New York

The status bar at the bottom indicates "Total rows: 2" and "Query complete 00:00:00.109".

(b) Management needs to confirm the size of the account portfolio. Calculate the total number of accounts and name the result total_accounts

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including schemas, aggregates, collations, domains, FTS configurations, dictionaries, parsers, templates, foreign tables, functions, materialized views, operators, procedures, sequences, tables, trigger functions, types, views, subscriptions, and roles. The main query editor shows the following SQL code:

```

/*
(b) Management needs to confirm the size of the account portfolio. Calculate
the total number of accounts and name the result total_accounts. (5 marks)
*/

-- Your answer here
select count(account_id) as total_accounts
from accounts;

-- Expected result: 15

```

The query results are displayed in the Data Output tab, showing one row:

total_accounts
15

The status bar at the bottom indicates "Total rows: 1" and "Query complete 00:00:00.107".

Question 3 – Account Balance Analysis (20 marks)

(a) The Finance Manager wants to monitor funds held in Checking accounts. Calculate their total balance and name the result total_checking_balance. (10 marks)

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'public' schema and various objects like aggregates, collations, domains, and tables. The main pane shows a SQL query editor with the following content:

```

79
80
81 /*
82 QUESTION 3 - ACCOUNT BALANCE ANALYSIS (20 marks)
83 (a) The Finance Manager wants to monitor funds held in checking accounts.
84 Calculate their total balance and name the result total_checking_balance.
85 (10 marks)
86 */
87
88 -- Your answer here
89 select sum(balance) as total_checking_balance
90 from accounts
91 where account_type = 'Checking';
92
93 -- Expected result: 31000.00
94
95

```

The query is executed, and the results are displayed in the 'Data Output' pane. The results show a single row with the total balance for checking accounts.

total_checking_balance
31000.00

The status bar at the bottom indicates: 'Total rows: 1 Query complete 00:00:00.112' and 'Successfully run. Total query runtime: 112 msec. 1 rows affected.'

(b) For each customer living in Los Angeles, display customer_id, full_name, and total_balance across all account types. Sort by total_balance from highest to lowest. (10 marks)

The screenshot shows the pgAdmin 4 interface with a SQL query editor and a data output window. The query is as follows:

```
-- Your answer here
select c.customer_id, c.first_name || ' ' || c.last_name as full_name, sum(a.balance) as total_balance
from customers c
join accounts a on c.customer_id = a.customer_id
where city = 'Los Angeles'
group by c.customer_id
order by total_balance desc;
```

The data output window shows the following results:

customer_id (PK) integer	full_name text	total_balance numeric
1	5 MichaelLee	60000.00
2	6 JenniferWa...	15000.00

The status bar indicates: Total rows: 2, Query complete 00:00:00.116, Successfully run. Total query runtime: 116 msec. 2 rows affected.

Question 4 – Branch and Customer Portfolio Analysis (20 marks)

(a) Identify the branch with the highest average account balance. Display branch_id, branch_name, city, and average_balance. Round to two decimal places and include ties. (10 marks)

The screenshot shows the pgAdmin 4 interface with a SQL query editor and a data output window. The query is as follows:

```
-- Your answer here
SELECT
  a.branch_id,
  b.branch_name,
  c.city,
  ROUND(AVG(a.balance)::numeric, 2) AS average_balance
FROM accounts a
JOIN customers c ON c.customer_id = a.customer_id
JOIN branches b ON b.branch_id = a.branch_id
GROUP BY a.branch_id, b.branch_name, c.city
ORDER BY average_balance DESC
LIMIT 1;
```

The data output window shows the following results:

branch_id integer	branch_name character varying (50)	city character varying (50)	average_balance numeric
1	14 North Beach	Los Angeles	30000.00

The status bar indicates: Total rows: 1, Query complete 00:00:00.110, Successfully run. Total query runtime: 110 msec. 1 rows affected.

(b) Identify the customer who owns the single account with the highest current balance. Display customer_id, full_name, account_id, account_type, and balance. Include ties. (10 marks)

The screenshot shows the pgAdmin 4 interface with a query window open. The query is as follows:

```

144 account with the highest current balance. Display customer_id, full_name,
145 account_id, account_type, and balance. Include ties if any. (10 marks)
146 */
147
148 -- Your answer here
149 select c.customer_id, c.first_name || ' ' || last_name as full_name, a.account_id, a.account_type, a.balance
150 from accounts a
151 join customers c on c.customer_id = a.customer_id
152 group by c.customer_id, full_name, a.account_id, a.account_type, a.balance
153 order by balance desc
154 limit 1
155 ;
156
157 -- Expected result:
158 -- 5 | Michael Lee | 10 | Savings | 50000.00
159
160 /*
  
```

The results pane shows the following data output:

customer_id	full_name	account_id	account_type	balance
5	Michael Lee	10	Savings	50000.00

The status bar at the bottom indicates: "Successfully run. Total query runtime: 107 msec. 1 rows affected." The window title is "pgAdmin 4" and the connection is "danghuynhquynhnhu/postgres@PostgreSQL 18".

Question 5 – Customer Value and Activity (20 marks)

(a) Identify the most active customers based on the total number of transactions across all their accounts. Display customer_id, full_name, and total_transactions. Include ties. (10 marks)

The screenshot shows the pgAdmin 4 interface with a SQL query editor and a data output window. The query is as follows:

```

-- Your answer here
select c.customer_id, c.first_name || ' ' || c.last_name as full_name, count(t.transaction_id) as total_transactions
from accounts a
join customers c on a.customer_id = c.customer_id
join transactions t on a.account_id = t.account_id
group by c.customer_id
order by total_transactions desc
fetch first 1 rows with ties

```

The data output window shows the following results:

customer_id	full_name	total_transactions
1	AliceJohnson	4
2	JaneDoe	4

The status bar indicates: Total rows: 2, Query complete 00:00:00.161, Successfully run. Total query runtime: 161 msec. 2 rows affected.

(b) Identify the customer with the highest combined balance across Checking and Savings accounts. Exclude Credit Card accounts. Display customer_id, full_name, and total_deposit_balance. Include ties. (10 marks)

The screenshot shows the pgAdmin 4 interface with a SQL query editor and a data output window. The query is as follows:

```

-- Your answer here
select c.customer_id, c.first_name || ' ' || c.last_name as full_name, a.customer_id,
SUM(a.balance) AS total_deposit_balance
FROM customers c
JOIN accounts a ON c.customer_id = a.customer_id
WHERE account_type IN ('Checking', 'Savings')
GROUP BY c.customer_id, a.customer_id
ORDER BY total_deposit_balance DESC
fetch first 1 rows with ties;
-- Expected result:
-- 5 | Michael Lee | 60000.00

```

The data output window shows the following results:

customer_id	full_name	customer_id	total_deposit_balance
5	MichaelLee	5	60000.00

The status bar indicates: Total rows: 1, Query complete 00:00:00.106.

Question 6 – Advanced Finance Analysis (20 marks)

(a) Identify the branch with the highest total balance across all account types. Display branch_id, branch_name, and total_balance. Include ties. (10 marks)

The screenshot shows the pgAdmin 4 interface with a SQL query executed. The query is as follows:

```

-- Your answer here
WITH ranked_branches AS (
  SELECT
    b.branch_id,
    b.branch_name,
    SUM(a.balance) AS total_balance,
    DENSE_RANK() OVER (ORDER BY SUM(a.balance) DESC) AS rnk
  FROM public.branches b
  JOIN public.accounts a ON b.branch_id = a.branch_id
  GROUP BY b.branch_id, b.branch_name
)
SELECT
  branch_id,
  branch_name,
  total_balance
FROM ranked_branches
WHERE rnk = 1;

```

The result set shows one row:

branch_id	branch_name	total_balance
14	North Beach	60000.00

The interface also shows the 'Data Output' tab with the same result set. The status bar indicates 'Total rows: 1' and 'Query complete 00:00:00.212'.

(b) Rank all customers by total balance across all account types. Equal totals must receive the same rank without gaps. Display customer_id, full_name, total_balance, and balance_rank. Do not use a CTE. (5 marks)

Query

```
-- 14 | North Beach | 60000.00
--
/*
(b) Rank all customers by total balance across all account types. Equal totals
must receive the same rank without gaps. Display customer_id, full_name,
total_balance, and balance_rank. Do not use a CTE. (5 marks)
*/
-- Your answer here
SELECT
  c.customer_id,
  CONCAT(c.first_name, ' ', c.last_name) AS full_name,
  SUM(a.balance) AS total_balance,
  DENSE_RANK() OVER (ORDER BY SUM(a.balance) DESC) AS balance_rank
FROM public.customers c
JOIN public.accounts a ON c.customer_id = a.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY balance_rank ASC;
-- Expected results:
-- 5 | Michael Lee | 60000.00 | 1
```

Data Output

	customer_id [PK] integer	full_name text	total_balance numeric	balance_rank bigint
1	5	Michael Lee	60000.00	1
2	4	Alice Johns...	25000.00	2
3	3	Bob Smith	20500.00	3
4	6	Jennifer Wa...	15000.00	4
5	2	Jane Doe	11500.00	5

Total rows: 6 Query complete 00:00:00.283

(c) Use a CTE to calculate the total number of transactions for every branch, including branches with no transactions. Return the branch or branches with the highest total. Display branch_id, branch_name, and total_transactions. (5 marks)

Query

```
-- Your answer here
WITH branch_transaction_counts AS (
  SELECT
    b.branch_id,
    b.branch_name,
    COUNT(t.transaction_id) AS total_transactions,
    DENSE_RANK() OVER (ORDER BY COUNT(t.transaction_id) DESC) AS rnk
  FROM public.branches b
  LEFT JOIN public.accounts a ON b.branch_id = a.branch_id
  LEFT JOIN public.transactions t ON a.account_id = t.account_id
  GROUP BY b.branch_id, b.branch_name
)
SELECT
  branch_id,
  branch_name,
  total_transactions
FROM branch_transaction_counts
WHERE rnk = 1;
```

Data Output

	branch_id [PK] integer	branch_name character varying (50)	total_transactions bigint
1	8	South Bay	4
2	1	Main	4

Total rows: 2 Query complete 00:00:00.512

Link gifhub:

<https://github.com/NhuqDangg/MIS-443---Business-Data-Management/tree/main>

Mark Distribution

Question	Marks
Question 1	10
Question 2	10
Question 3	20
Question 4	20
Question 5	20
Question 6	20

END OF EXAMINATION